EE2016 Microprocessor Theory & Lab
Aug – Nov 2025

Week7: Control Unit Anatomy & Design:
Hardwired Control Unit Design

Dr. R. Manivasakan, OWSM Lab, EE Dept, IIT, Madras

Copy righted material. Only for private use by students of EE2016F25. Not for public release.

Manual Execution of a Program

- Manual execution of program (not automated)
  - We manually identify the next instruction to be executed, drive the EU using the right control signals
- Automation is done by CU.
  - Uses the PC (part of CU) to hold address of next instruction to be executed.
  - And also the PFCL would also automatically track the next instruction to be executed even of the sequence is out of order
  - No human intervention is needed. Automatic.
  - In general, for all subengines in EU (even in CU), hardwired CU gives control signals at the right datapath point, in the right time, to get the right intermediate output, resulting in the final desired output.

Control Unit Design

- We have designed an EU so far and learned how to control EU using control lines: we, the human being interpreting the program, generate control signals and drive the EU
  - Now, let us learn how to automate the program to drive the EU

We have so far learned .....

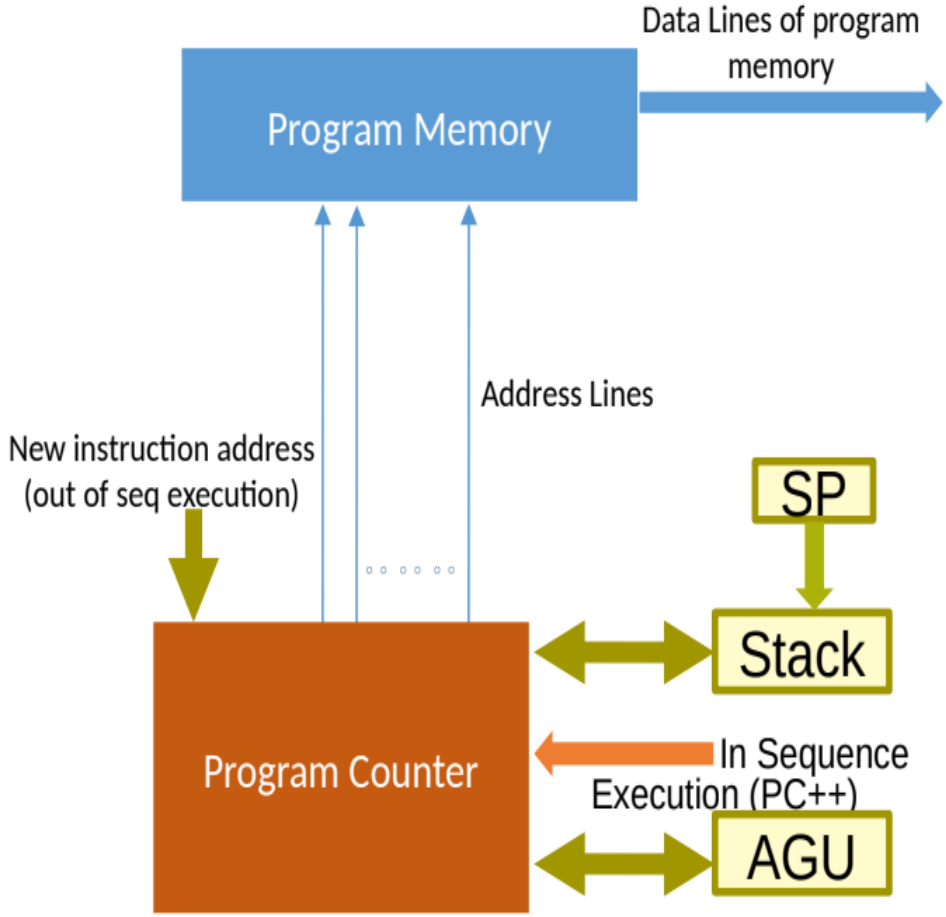
- How to design EU of a processor?
- How to drive EU using Control Signals?
- Running an assembly program
  - involves generating the appropriate control signals given a program (and instructions thereof)
  - The generated control signals now drive the EU, at the right datapath point, in the right time, to get the right intermediate output, resulting in the final desired output.
  - Normally, the program is executed in sequence.
  - Occasionally change sequence (program flow) as per the outcome of previous instruction(s).
- Next we will learn
  - How to automate the controller (so far a human being a controller) using a Control Unit and Program Memory

Control Unit

- **Control unit** (CU) uses a Program to control/drive the operation of all sub-units in muP (esp. the Execution unit (EU) of the processor apart from controlling peripherals including memory)
  - Each program instruction (Execution Unit Control Instruction or EUCI) is executed as a series of steps in sequence driven by a state machine
    - Drives data-flow and ALU instructions through control signals for EU
    - We can say that the control unit takes charge of running the given program, all by itself: Automates the execution of a program (including CU itself), across datapath & across clock interval.
    - Implemented using a binary decoder to convert assembly instructions into timing & control signals that direct the operation of the EU, I/O and memory.
      - Dictates what has to be done in each clock period.
  - Program is normally executed in Sequence, but some instructions can modify the flow of the program (Program Flow Control Instruction) – rather the outcome of some instructions (conditional statements) might alter the course of the program: CU modifies PC value, etc
- CU Components: Instruction Register (IR), Instruction Buffer Register (IBR), Program Counter (PC), Stack Pointer (SP), Instruction Decoder (implemented as Execution Unit Control Logic (EUCL) and Program Flow Control Logic (PFCL))
- Out of the above, Instruction Decoder is to be designed and rest of them are just registers.

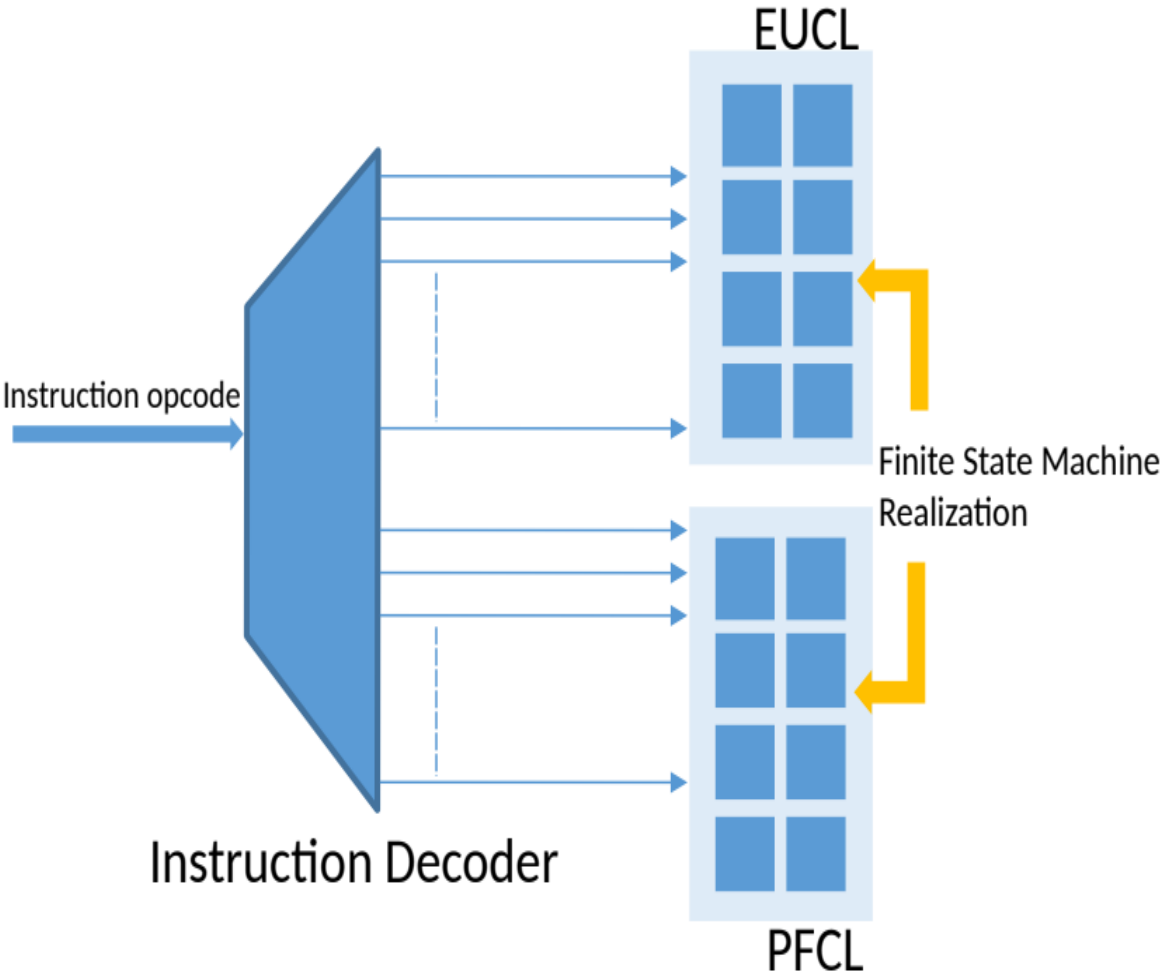
Program Counter (PC)

- PC holds the address of the next instruction to be executed and drives the address lines of PM
- Instructions are usually fetched sequentially from memory (hence PC is usually incremented after each instruction is executed)
  - but **program flow control instructions** might change the sequence by loading a new value in PC



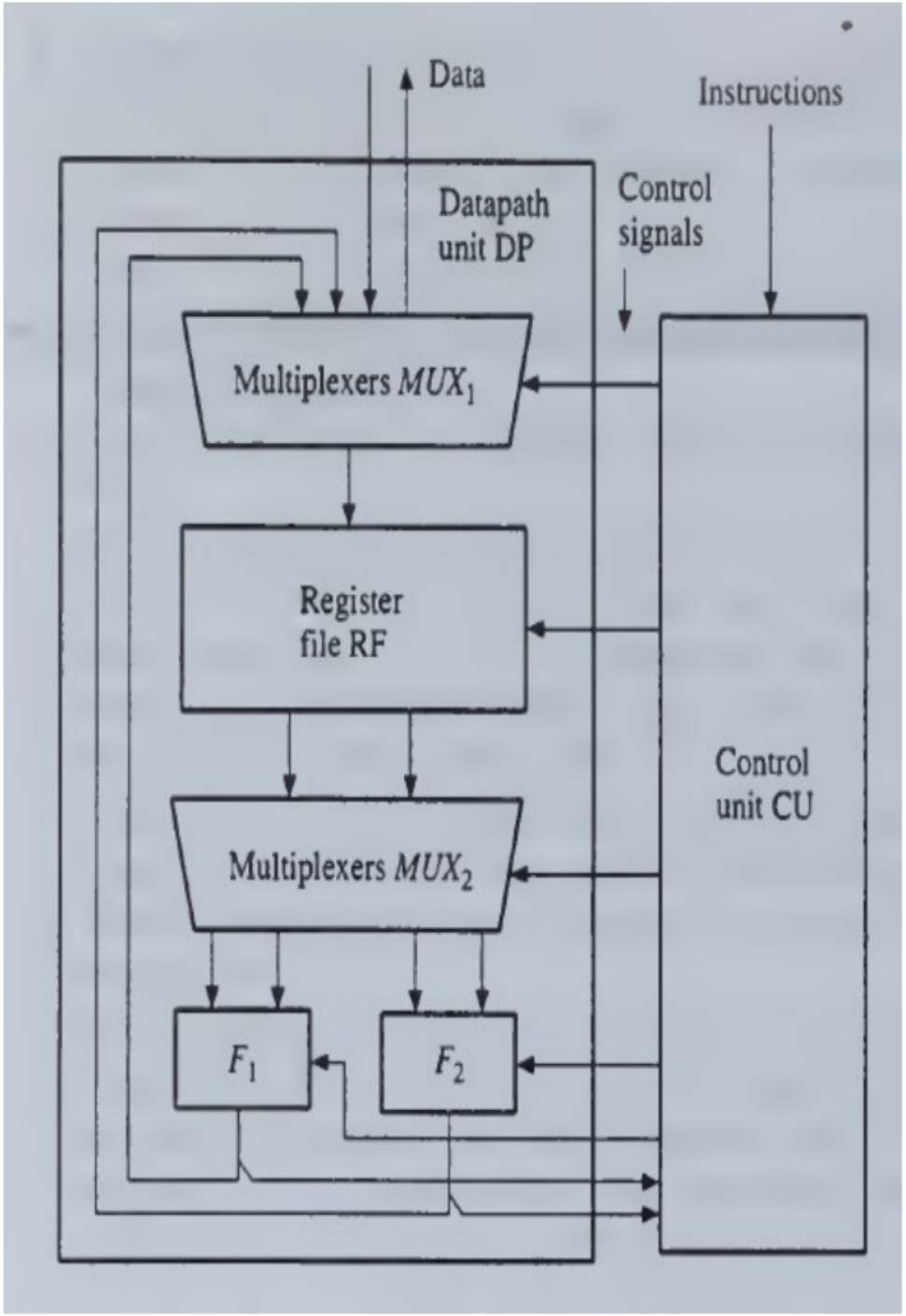
EUCL, PUCL and Instruction Decoder

- An Instruction Decoder decodes the op-code and drives the state machine corresponding to the op-code
  - Execution Unit Control Logic (EUCL): issues control signals for the execution unit
- Control Unit consists of sequential logic circuit or finite state machine that generates specific sequences of control signals in response to externally supplied instructions



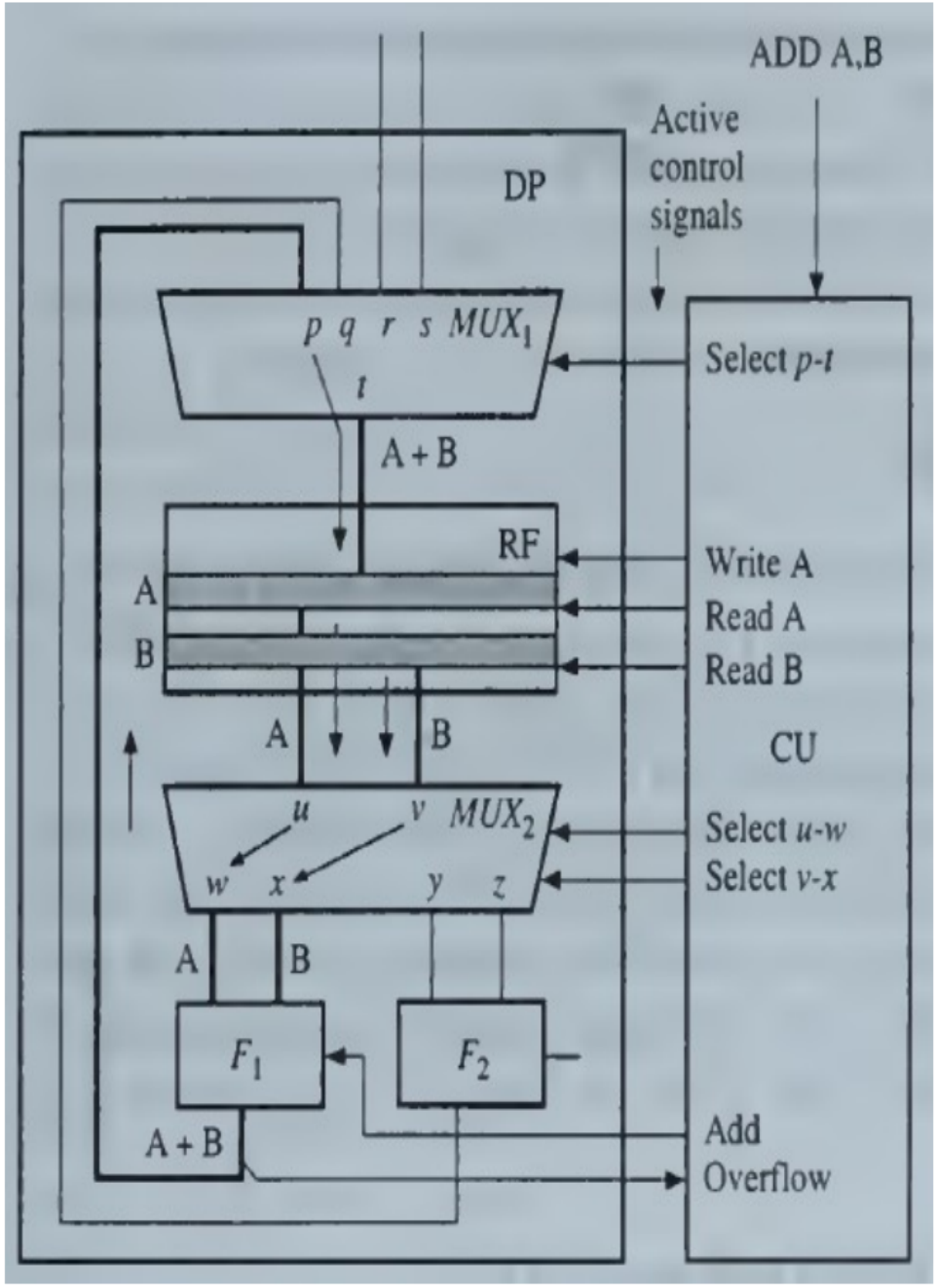
Generic Model of Datapath (DP) Unit & Associated Control Unit

- Datapath is a network of functional & storage units capable of doing some (micro) operations on the data words
  - eg. contains circuits to arithmetic and logical operations on (fixed pt & floating pt) words
- Purpose of CU --> issue control signals to the datapath
- Control signals enter the datapath at the ‘control’ points
  - Select functions to be performed on data at specific times and route the result to right parts /points in the datapath
  - In essence control signals reconfigures the datapath to implement a specified instruction or program.
- Datapath hardware framework
  - Register file to store temporarily data
  - Two functional units to process data
  - Multiplexers to steer data to right parts of datapath circuit for processing
- Register File
  - A group of files in which concurrently reading two different registers are possible
- S



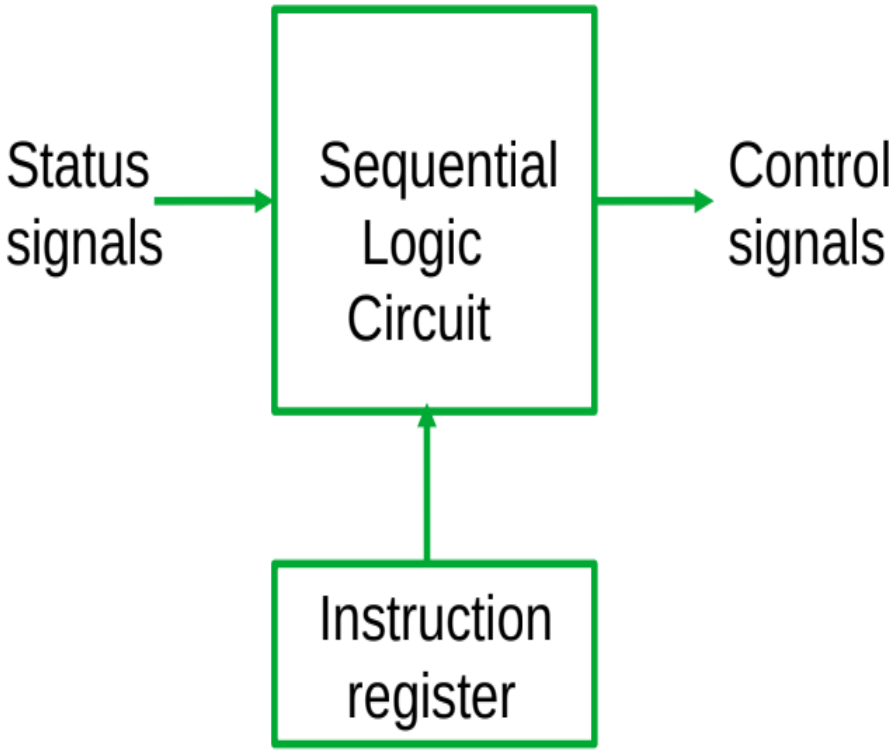
Control Unit Design: Motivation Thr Eg. ADD A,B

- Fig 5.1 & 5.2
  - Function select: Add
  - Storage control: Read A, Read B, Write A
  - Data routing: Select p-t, select u-w, select v-x
- A simpler micorprocessor model
  - Showing addition of two operands ADD A, B (A:=A+B)
  - Registers of RF can be read from and written into, in the same clock cycle (edge triggered FFs)
- Control Unit must implement:
  - Load A, B in the data registers in RF (GPRs)
  - ADD operation to be performed on data
  - Control signal to write the result
  - Route the logical connections through the multiplexers
- CU must generate following three types of signals
  - Function select: ADD
  - Storage control: Read A, Read B and Write A
  - Data routing: Select p-t, u-w, y - x
- S



Control Unit Design: Hardwired

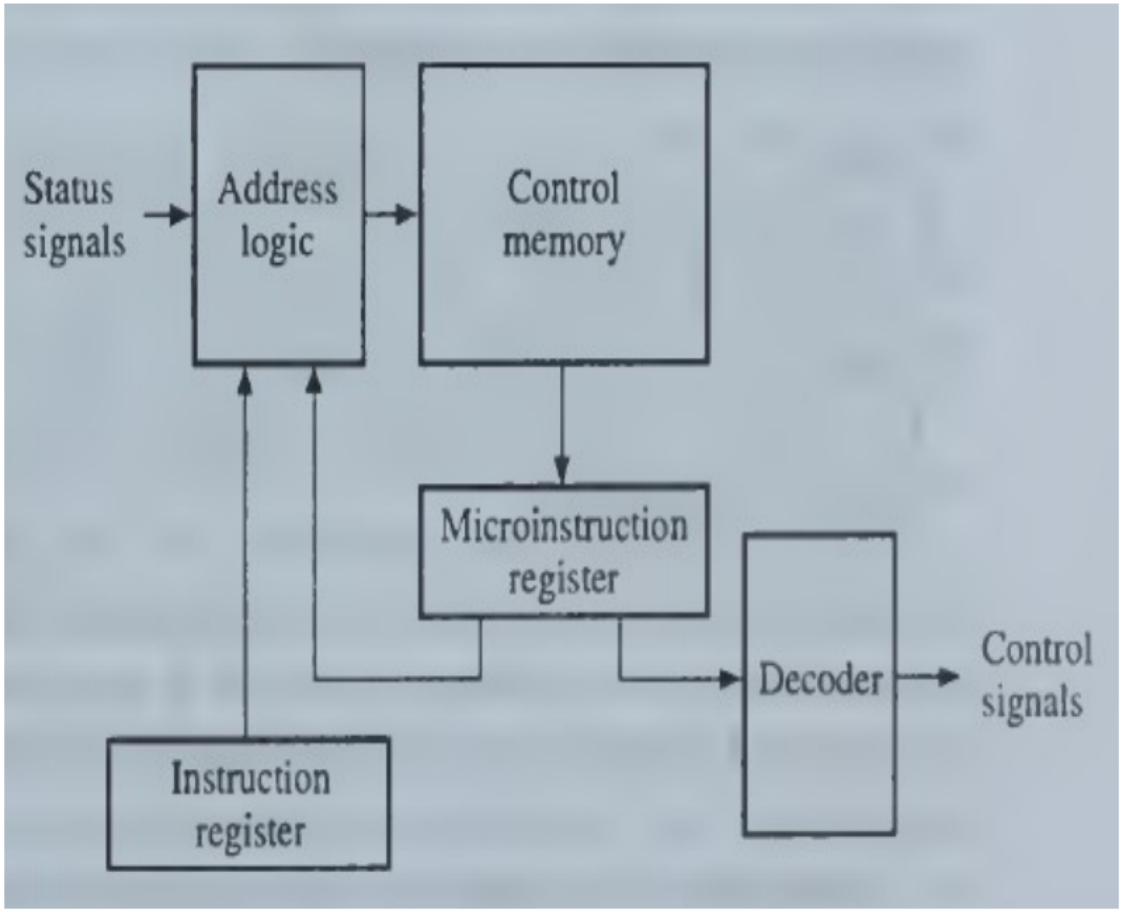
- Control Unit Implementation
  - Hardwired
  - Microprogrammed
- Hardwired
  - As a mix of sequential & combinational circuit which generates low level (may not be always digital) sequence of control signals, given an instruction & for all instructions of a given program
- Advantages
  - Minimum number of components used
  - faster
- Disadvantages
  - Inflexible (once designed & implemented can't be changed)



- John P. Hayes, (1978) Computer Architecture and Organization McGraw-Hill, 1978
- Chapter 3 pp (303 - 312)

Control Unit Design: Microprogrammed

- Microprogrammed
  - It is a ‘look-up’ table kind of implementation of mapping from assembly instructions to low level control signals.
  - This mapping is kept in ‘control’ memory
- Advantages
  - Simple (easier to design, debug & implement)
  - flexible
- Disadvantages
  - slower



Control Unit Design: Hardwired Control

- To build circuits which would map instructions into low level control (digital) signals
  - Hardwired CU design involves
    - Power consumed
    - Amount of hardware (logical blocks)
    - Cost involved in design process
    - Latency
- Method 1: Classical method
- Method 2: one-hot method

- Control Unit Implementation for the whole processor is most complex
  - No single acceptable design method yet
  - Several linked independent designed sequential circuits – optimal solution in practice
- Finite state machine (FSM): State table characterization --> sequencing --> implemented by sequential circuit
- S

Hardwired Control Design: Classical Method

- Mealy type
- Output depends on current state & current input
- Next state depends on current state and current input

State	I1	I2	....	Im
S1	S1,1 O1,1			
S2				
Sn				Snm,Onm

Hardwired Control Design: Classical Method

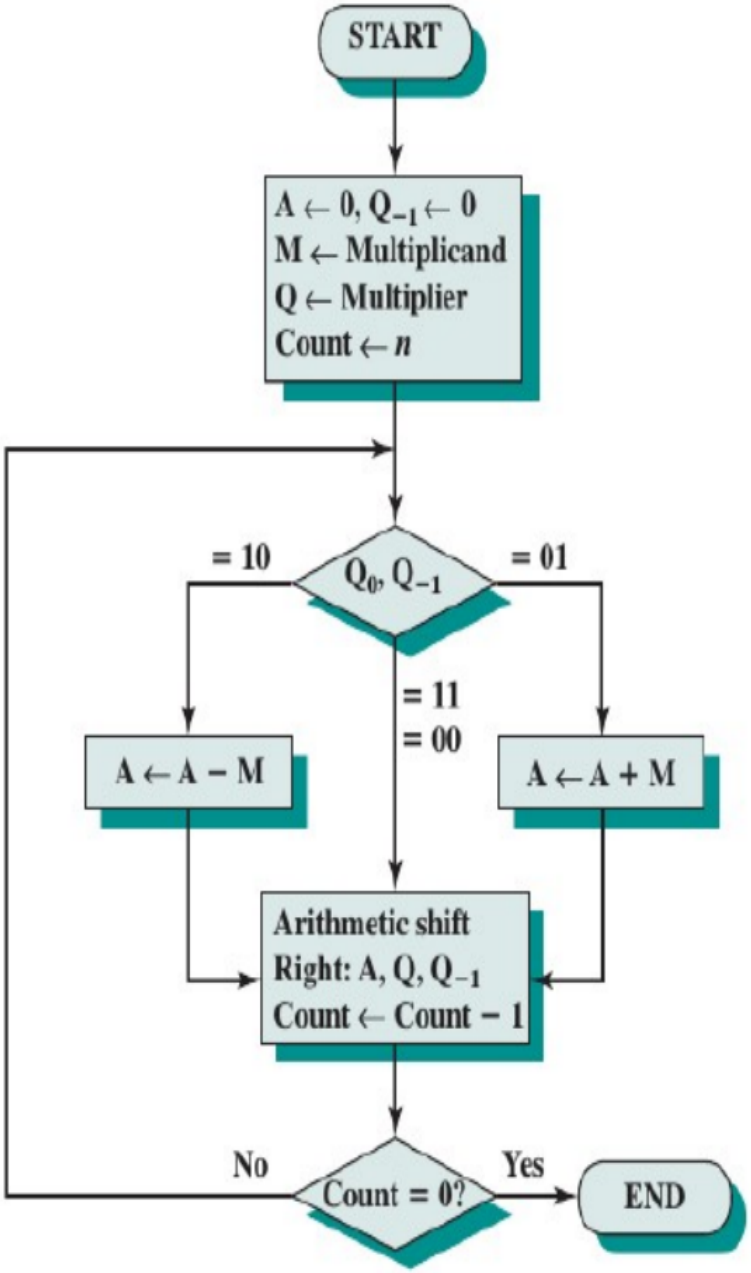
- Moore type
  - S
- Output depends on current state only (not on current input)
- Next state depends on current input and current state

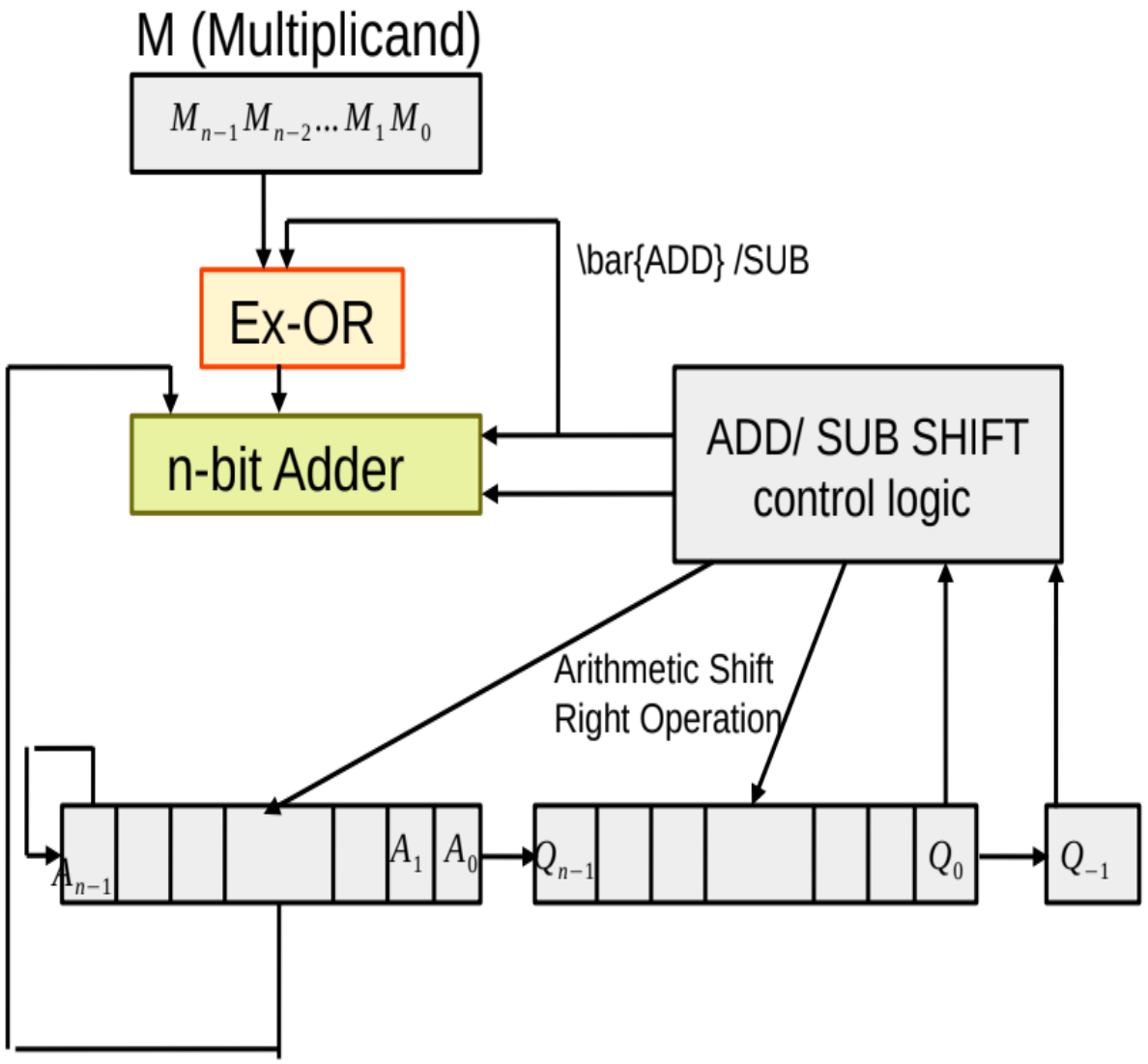
State \ Inpt	I1	I2	....	Im	Outputs
S1	S1,1				O1
S2					
Sn				Sn,m	On

Control Unit Design: Instruction Decoder

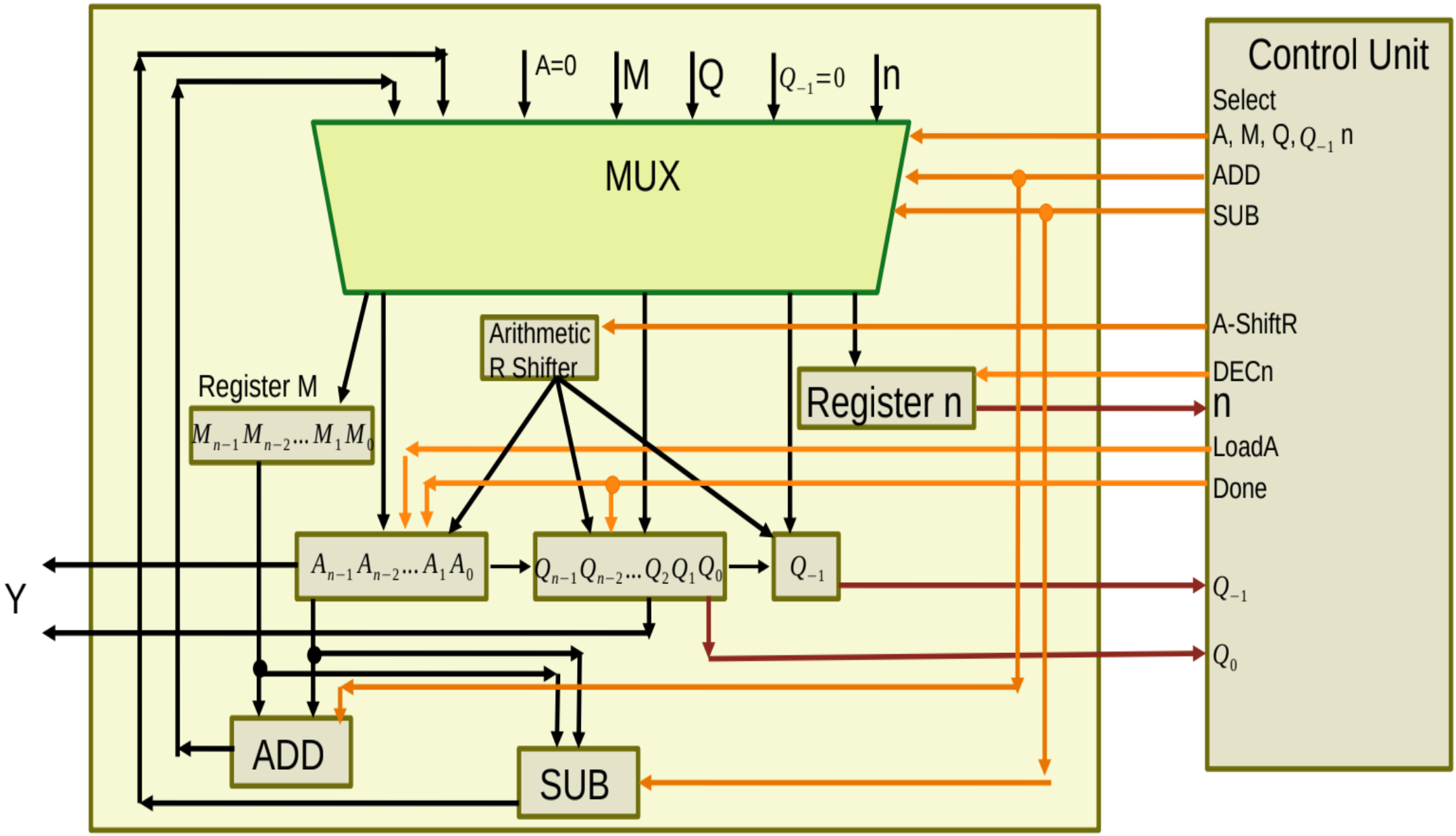
- General H/W structure of Control Units
  - IR, IBR, PC, MAR, Stack, SP, ID
  - Only Instruction Decoder has to be designed (combinational + sequential)
- Purpose of CU is to
  - map the programmer's instructions / commands to a sequence of low level control signals
  - Apply the above low level control signals to all sub-engines of muP
  - Esp. elements in EU, along the datapath at the right time & right digital circuit point.
- S
- Ideally, the control hardware (in hardwired Control) be included in the CU and all computations (arithmetic / logical) are implemented in EU
  - Hypothetical
- But in actual design, the control (hardwired CU) associated with the sub-engine is tied up along with the sub-engine's hardware. Examples:
  - ALU 74181 (ADD, SUB, ..)
  - the Booth's multiplier (MULT)
  - FPU
- Hardwired implementation of every instruction is composed of two parts:
  - Data path hardware implementation
  - Control hardware implementation controlling the above execution element

Booth's Multiplier: Algorithm, Hardware Design & corresponding Control Unit Design





Booth's Multiplier: Algorithm, Hardware Design & corresponding Control Unit Design



State Table for Q0Qn-1

State	(10) (n> 0)	(00, 11) n>0	(01) n>0	XX n=0	Add	Sub	AShR	Decn	LoadA	Select AMQ	Done
S0 (Start)	S1	S2	S3	S5	0	0	0	0	0	1	0
S1 (Sub)	S2	S2	S2	S5	0	1	0	0	1	0	0
S2 (AShR)	S4	S4	S4	S5	0	0	1	0	0	0	0
S3 (Add)	S2	S2	S2	S5	1	0	0	0	1	0	0
S4 (Dec)	S1	S2	S3	S5	0	0	0	1	0	0	0
S5 (Done)	S5	S5	S5	S5	0	0	0	0	0	0	1

Example 2 - CU Design for GCD Processor (Hypothetical)

- GCD(X,Y)
  - Largest integer which divides both X & Y without reminder
  - Eg gcd(12,16) = 4
  - Assumed gdc(0,0) = 0
- Algorithm (Variation of Euclid's Algorithm)

```
gcd(Input X,Y; out Z)
Register XR, YR, TEMP;
XR = X;
YR=Y;
while(XR>0) do begin
        If (XR<=YR) then begin; (Swap  YR & XR)
          TEMP := YR;
          YR = XR;
          XR = TEMP;
          end
        XR = XR – YR; (Subtract YR from XR)
        End
Z = YR (Output)
End gcd
```

Example Illustration of Algorithm for GCD

- GCD(X,Y)
  - Largest integer which divides both X & Y without reminder
  - Eg gcd(12,16) = 4
  - Assumed gdc(0,0) = 0
-

Conditions				
		XR:=20;YR:=12;		
XR>0	XR>YR	XR:=XR-YR=8;		
XR>0	XR<=YR	YR=8;XR=12;		XR:=XR-YR=4;
XR>0	XR<=YR	YR=4;XR=8;		XR:=XR-YR=4;
XR>0	XR<=YR	YR=4;XR=4;		XR:=XR-YR=0;
XR<=0		Z=4;		

CU Design for GCD Processor: Procedure Outline

- We define states in the variant of Euclid's algo
  - State 0 – S0 input data / initialization
  - State 1 – S1 swap (only if certain conditions are satisfied)
  - State 2 – S2 subtract (only if certain condition is statisfied)
  - State 3 – S3 output the result (Output available, flagged by "Done" signal)
- Also, the input (digital) signals to CU are the results of comparisons (XR>0) and (XR<YR) which are again Boolean.
- From modified Euclid algo for GCD(X,Y), one infers
  - (a) At state S0, if input condition (XR>0) independent of the other, then input 0* which means, control would not enter loop reaching S3 (next state)
  - (b) Again at (current) state S0, if input is 10 then S1 (swap) would happen
  - (c) At state S0, if input is 11, then S2 (subtract) would happen
  - Correspondingly, at S0, subtract = 0, swap = 0, selectXY = 1, LoadXR = 1, LoadYR = 1
  - The above completes 1st row. Other three rows are straight forward to understand.
- Put the above observations in the form of table, state table, excitation table (TT) and finally for each output, one can use Karnaugh map to derive expression & draw the digital circuit.

CU Design for GCD Processor: State Table

State	Input (XR>0) (XR>=YR) dgtl sgnls								
	0*	10	11	Subtract	Swap	Select XY	Load XR	Load YR	Done
S0 (Begin)	S3	S1	S2	0	0	1	1	1	0
S1 (Swap)	S2 (NA)	S2	S2 (NA)	0	1	0	1	1	0
S2 (Subtract)	S3 (NA)	S1	S2	1	0	0	1	0	0
S3 (End)	S3	S3	S3	0	0	0	0	0	1

- State Table
  - Given the current state, input digital signals, compute the next state and current control signals
- First row corresponds to S0. Depending on input digital signals, decide whether the next state is S1 or S2 or S3
- Second row, corresponds to current state being S1, when (XR>0)=0, how can the next state be S2?
- Also, 2nd row, 3rd col, when (XR<=YR)=0, how can the next state be S2?
- Third row, corresponds to current state being S2. When (XR>0)=0, how can the current state be S2?
- 4th row is straight forward to derive and is a dead state

CU Design for GCD Processor: Excitation Table

Inputs		Present State		Next State		Current control signals				
(XR>0)	(XR>=YR)	D1	D0	D1+	D0+	Subtract	Swap	Select XY	Load XR	Load YR
0	d	0	0	1	1	0	0	1	1	1
0	d	0	1	1	0	0	1	0	1	1
0	d	1	0	1	1	1	0	0	1	0
0	d	1	1	1	1	0	0	0	0	0
1	0	0	0	0	1	0	0	1	1	1
1	0	0	1	1	0	0	1	0	1	1
1	0	1	0	0	1	1	0	0	1	0
1	0	1	1	1	1	0	0	0	0	0
1	1	0	0	1	0	0	0	1	1	1
1	1	0	1	1	0	0	1	0	1	1
1	1	1	0	1	0	1	0	0	1	0
1	1	1	1	1	1	0	0	0	0	0

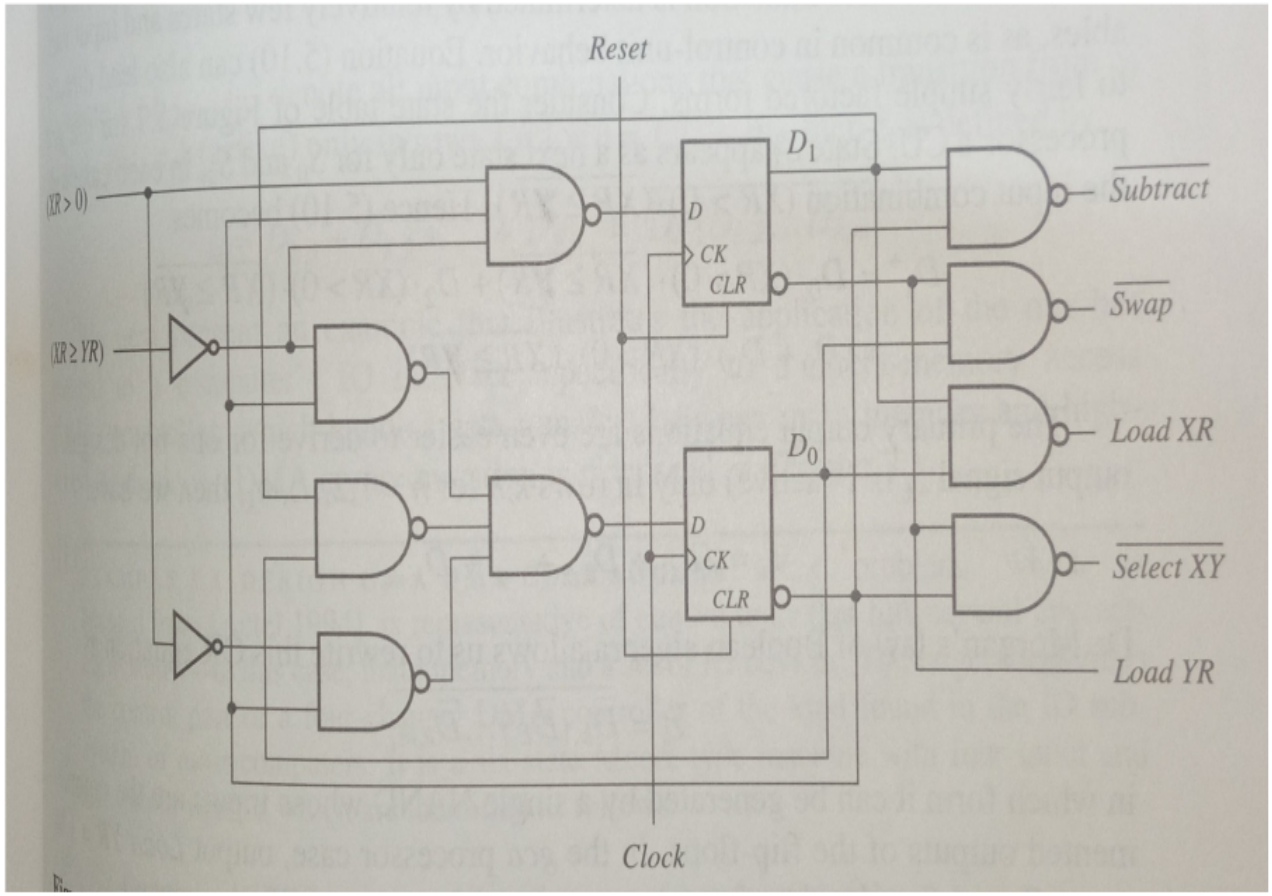
CU Design for GCD Processor: Excitation Table

- Excitation Table
  - Table which consists of inputs ((XR>0) & (XR<=YR)), present state (S0 – 00, S1 – 01, S2 – 10, S3 – 11), next state (S0, through S3), current outputs (digital control signals: subtract, swap, Select XY, Load XR, Load YR).
  - Given the state table, one can deduce the excitation table
- First row corresponds to input of 0* with present (input) state being S0 (D1,D0 = 00).
- The 2nd row, the present state is 01, for which the current control signals are ... ...
- 3rd row corresponds to
- 4th row, the present state being

- From the excitation table,
  - Which is inherently a truth table, derive the Boolean expression for output state, current control signals in terms of current state, current input
- Boolean expressions are:
  - D1+ = (bar{(XR>0)}+(XR>YR)+D0)
  - D0+ = D1D0+bar{(XR>=YR)}bar{D0} +
  - bar{(XR>0)}bar{D0}
  - Subtract = D1bar{D0}
  - Swap = bar{D1}D0
  - Select XY = bar{D1}bar{D0}
  - Load XR = bar{D0}+bar{D1}
  - Load YR = bar{D1}

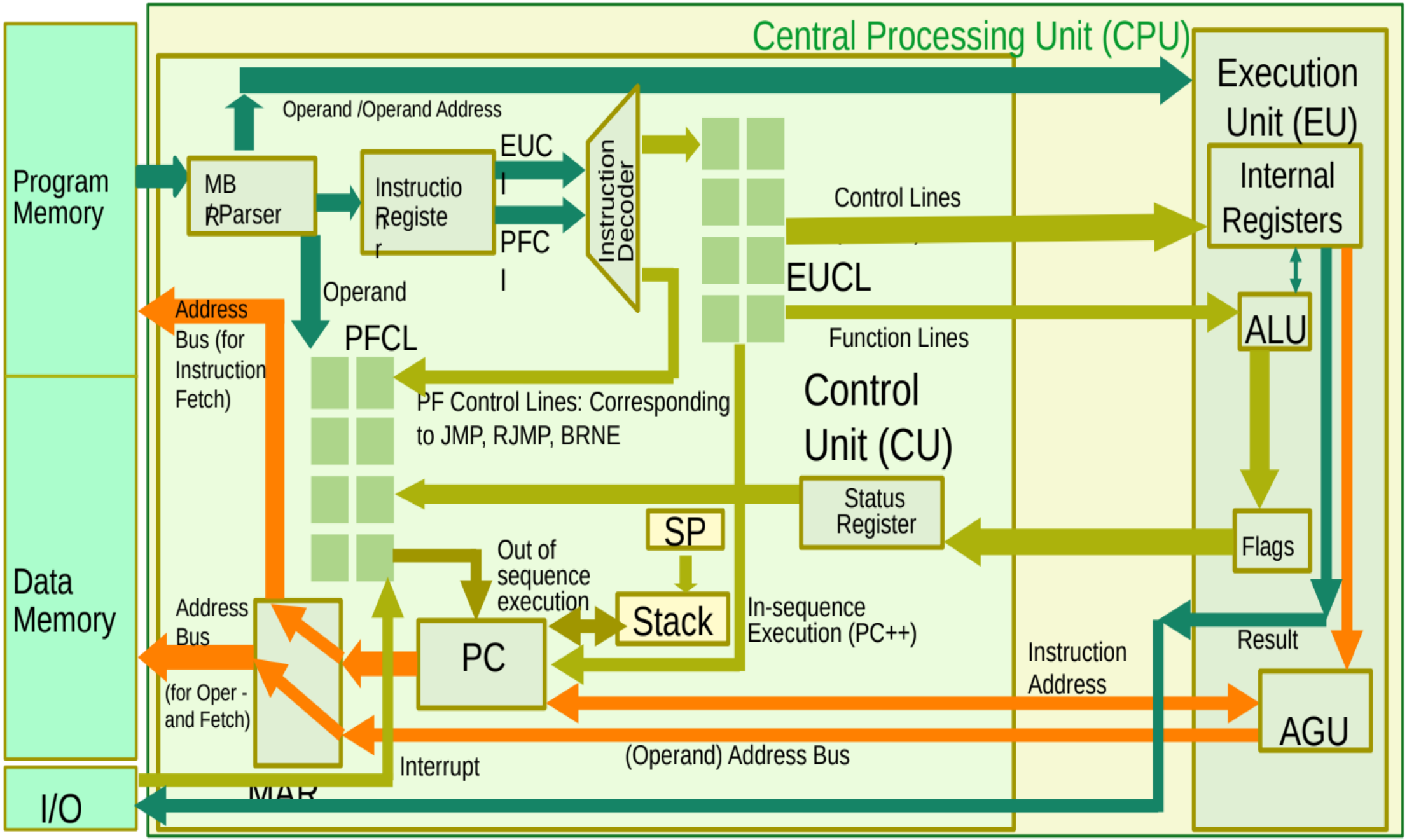
Circuit Realization of CU of GCD Processor

- GCD(X,Y)
  - Largest integer which divides both X & Y without reminder
  - Eg gcd(12,16) = 4
  - Assumed gdc(0,0) = 0
- Algorithm (Variation of Euclid's Algorithm)
- Hayes, Reproduced with due permission



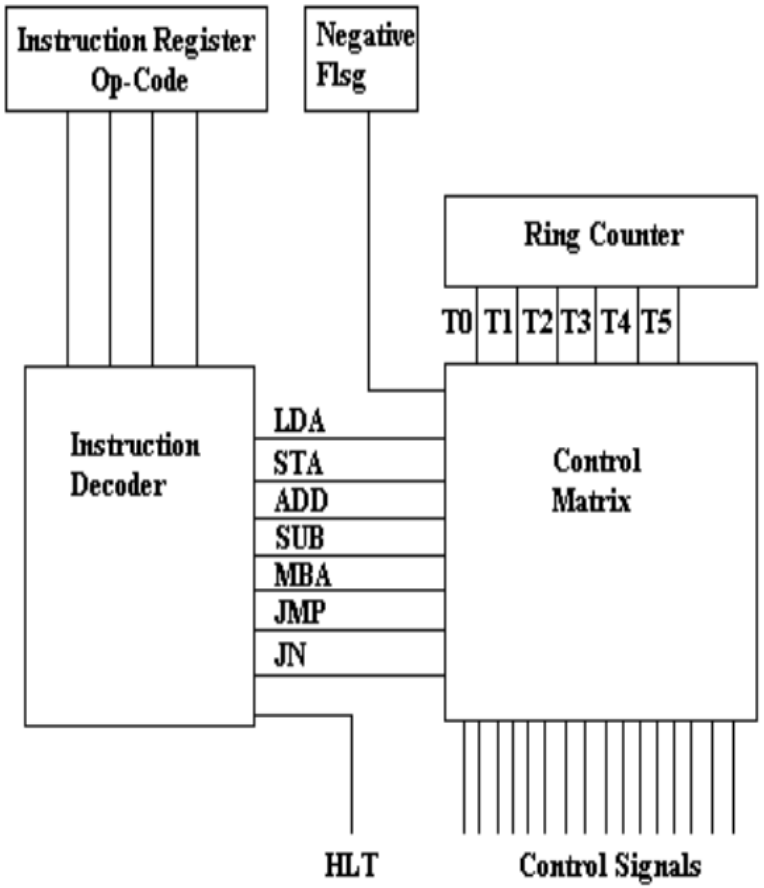
CU Design in General

- CU design for subunits in the execution unit is manageable complexity
- For instance, CU design for the Booths multiplier
- CU design for whole EU is complex
- Take away is that just like hardware implementation of execution unit (which is largely combinational circuits), so also the hardware implementation of CU.
  - But is implicit
  - Both combinational and sequential
  - Timed intervention in the data path
  - Control signals are at low level
  - Reasonably complicated as compared to implementation of data path or execution unit
- S


Central Processing Unit (CPU)
Program Memory
Data Memory
I/O
Execution Unit (EU)
Internal Registers
ALU
Flags
AGU
Operand /Operand Address
MB
R
Parser
Instructio
R
Registe
r
EUC
I
PFC
I
Instruction Decoder
Control Lines
EUCL
Function Lines
Operand
PFCL
Address Bus (for Instruction Fetch)
PF Control Lines: Corresponding to JMP, RJMP, BRNE
Control Unit (CU)
Status Register
SP
Out of sequence execution
Stack
In-sequence Execution (PC++)
Address Bus
(for Oper - and Fetch)
PC
Instruction Address
Result
Interrupt
(Operand) Address Bus
MAR

Hard Wired Control Unit

- Two Approaches for implementation of CU
  - Hardwired control unit
  - Microprogram control unit
- Hardwired control unit
  - Implemented through use of combinational logic units, which can generate the electrical control signals corresponding to an input
  - Generally faster than microprogrammed approach
  - Inherently not flexible. Fixed at the design stage itself.
- Microprogram control unit
  - Microprogram consists of microinstructions and stored in special control memory
  - A single instruction in assembly might consist of a microprogram (which consists of many microinstructions).
  - Much more flexible than hardware control unit.

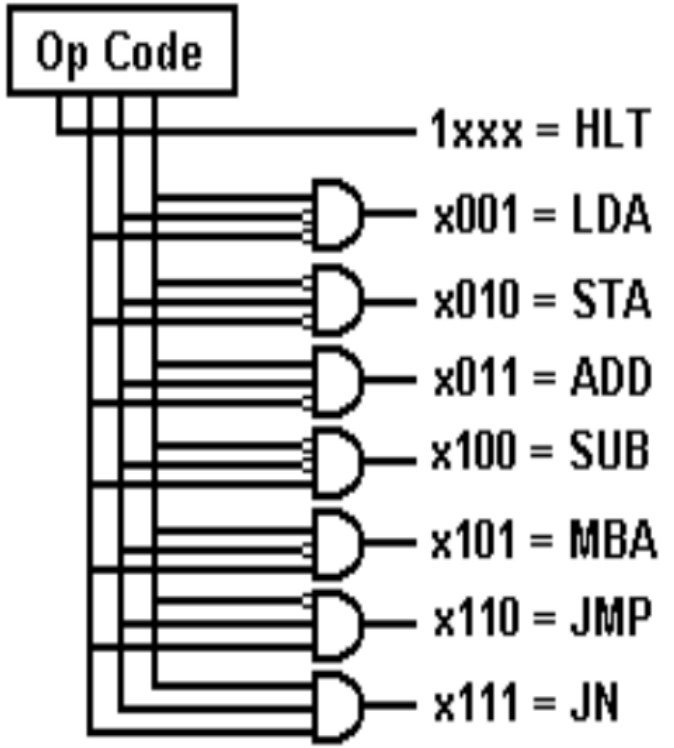


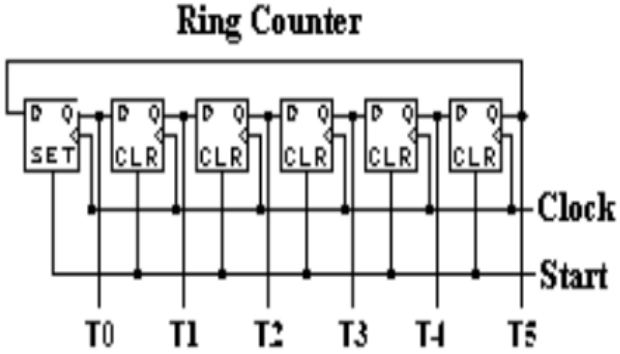
Constant length OP-Codes are assumed

https://illustratedman-code.github.io/HardVsMicro/

Instruction Decoder

- Instruction Decoder
  - Receives the opcode sent to it by the IR
  - Interprets the opcode as a specific signal
  - Sends a signal to the control matrix corresponding to the opcode from the IR
  - Each input line to the Control matrix maps to a different set of low-level control signals (a set of pins)
  - This set of low-level control signals implements all objectives of input opcode
- The above low-level control signals are put into action in sequence through a scheduling mechanism.
- Rather than a ring counter, a finite cyclic pulse train generator (scheduling mechanism), generated disjoint consecutive pulses
  - Whose edge triggers the operations of raw control signal (low level instructions)
  - Examples of low level instructions (raw control signals) are IP, LP, EP etc



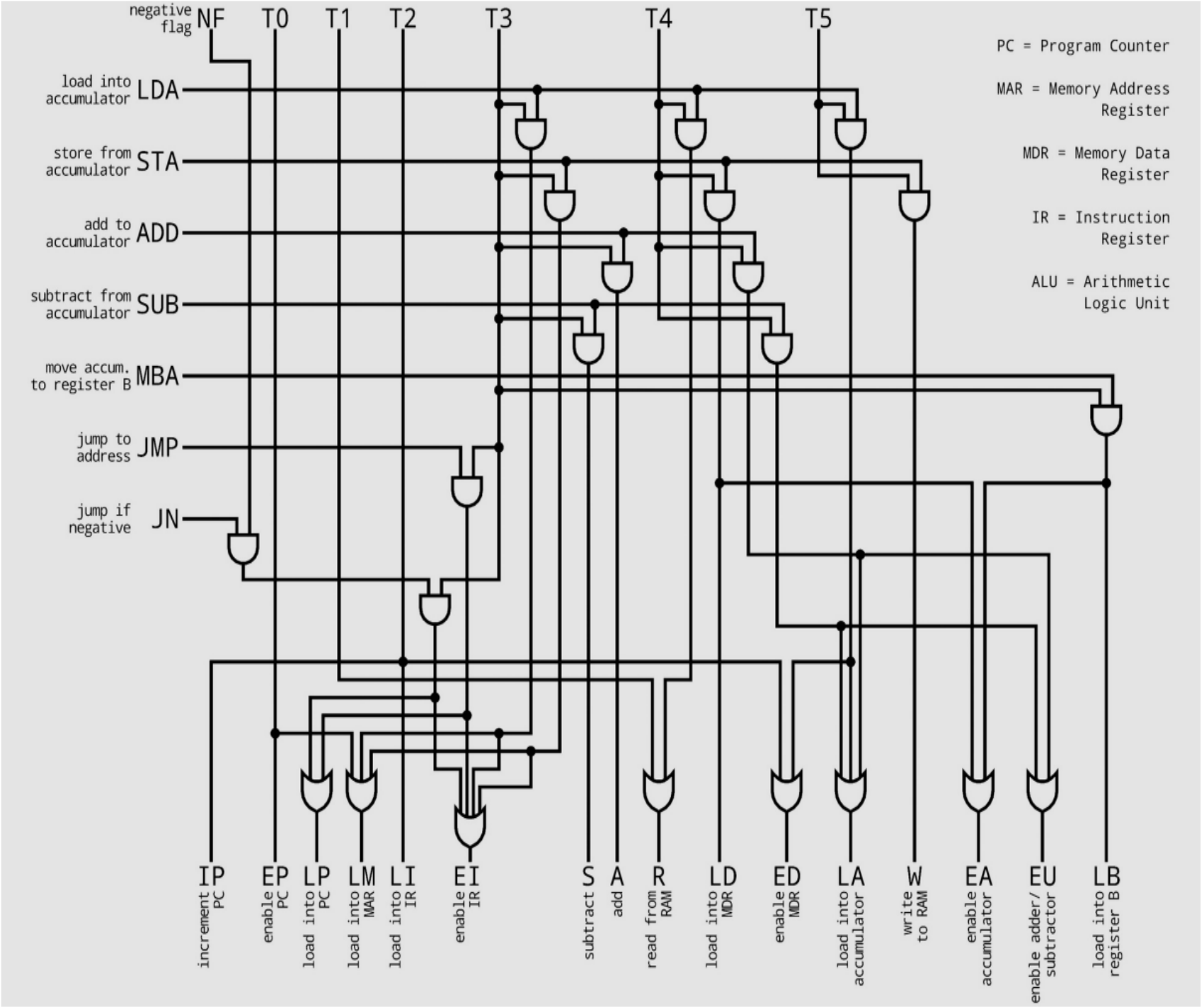


Hard Wired Control Unit - Matrix

- Low level instructions (raw control signals):
  - R, W, EP, IP, LP, EI, LI, S, A, EA, LA etc
- Examples of dissecting an AVR assembly instruction into low level instructions
- MOV Rd, Rs
  - Step 1: Read from source register ‘Rs’, its contents into the data bus, R – read from RAM
  - Step 2: Write the contents of data bus into the destination register ‘Rd’ – W - write into RAM
- ADD Rd, Rs
  - T4 (EU, LA), T3 (A)
-

Hardwired Control Unit

Matrix part



Hardwired Control Unit

- Mapping by control matrix
  - Contribution of each low-level signal to opcodes
  - Time sequencing

IP = T2
W = T5*STA
LP = T3*JMP + T3*NF*JN
LD = T4*STA
LA = T5*LDA + T4*ADD + T4*SUB
EA = T4*STA + T3*MBA
EP = T0
S = T3*SUB
A = T3*ADD
LI = T2
LM = T0 + T3*LDA + T3*STA
ED = T2 + T5*LDA
R = T1 + T4*LDA
EU = T3*ADD+T3*SUB
EI = T3*LDA + T3*STA + T3*JMP + T3*NF*JN
LB = T3*MBA

Hardwired Control Unit

Matrix part

Control Signal:	IP	LP	EP	LM	R	W	LD	ED	LI	EI	LA	EA	A	S	EU	LB
Instruction:																
"Fetch"	T2		T0	T0	T1			T2	T2							
LDA				T3	T4			T5		T3	T5					
STA				T3		T5	T4			T3		T4				
MBA												T3				T3
ADD											T4		T3		T4	
SUB											T4			T3	T4	
JMP		T3								T3						
JN		T3*NF								T3*NF						

Hardwired Control Unit

Matrix part

Table 1. An Instruction Set For The Basic Computer

Instruction Mnemonic	Op-Code	Execution Action	Register Transfers	Ring Pulse	Active Control Signals
LDA (Load ACC)	1	ACC<--(RAM)	1. MAR <-- IR 2. MDR <-- RAM(MAR) 3. ACC <-- MDR	3 4 5	EI, LM R ED, LA
STA (Store ACC)	2	(RAM) <--ACC	1. MAR <-- IR 2. MDR <-- ACC 3. RAM(MAR) <-- MDR	3 4 5	EI, LM EA, LD W
ADD (Add B to ACC)	3	ACC <-- ACC + B	1. ALU <-- ACC + B 2. ACC <-- ALU	3 4	A EU, LA
SUB (Sub. B from ACC)	4	ACC <-- ACC - B	1. ALU <-- ACC - B 2. ACC <-- ALU	3 4	S EU, LA
MBA (Move ACC to B)	5	B <-- ACC	1. B <-- A	3	EA, LB
JMP (Jump to Address)	6	PC <-- RAM	1. PC <-- IR	3	EI, LP
JN (Jump if Negative)	7	PC <-- RAM if negative flag is set	1. PC <-- IR if NF set	3	NF: EI, LP
HLT	8-15	Stop clock			
"Fetch"		IR <-- Next Instruction	1. MAR <-- PC 2. MDR <-- RAM(MAR) 3. IR <-- MDR	0 1 2	EP, LM R ED, LI, IP